

Resolving Errant GTID Issue in MySQL InnoDB Cluster Set

Overview

This document outlines a step-by-step resolution for an issue encountered while attempting to promote a DR-site as the primary cluster in a MySQL InnoDB Cluster Set setup. The operation failed due to the presence of errant transactions on one of the instances in the DR-site cluster.

Error Message

When checking the status of the InnoDB Cluster, you may encounter the following error:

***"transactionSetConsistencyStatus": "INCONSISTENT",
"transactionSetConsistencyStatusText": "There are 3 transactions that
were executed in this instance that did not originate from the
PRIMARY."***

```
MySQL mysql:33060+ ssl JS > clusterset.status()
{
  "clusters": {
    "DR-site": {
      "clusterErrors": [
        "ERROR: Errant transactions detected"
      ],
      "clusterRole": "REPLICA",
      "clusterSetReplicationStatus": "OK",
      "globalStatus": "OK_NOT_CONSISTENT",
      "status": "OK",
      "statusText": "Cluster is ONLINE and can tolerate up to ONE failure.",
      "transactionSetConsistencyStatus": "INCONSISTENT",
      "transactionSetConsistencyStatusText": "There are 1 transactions that were executed in this instance that did not originate from the PRIMARY."
    },
    "Prod-site": {
      "clusterRole": "PRIMARY",
      "globalStatus": "OK",
      "primary": "mysql:3306"
    }
  },
  "domainName": "POC_CS",
  "globalPrimaryInstance": "mysql:3306",
  "primaryCluster": "Prod-site",
  "status": "AVAILABLE",
  "statusText": "Primary Cluster available, there are issues with a Replica cluster."
}
```

Root Cause

An errant transaction is one that has been executed on a server but was not received via replication from the primary cluster. These can occur due to manual writes, misconfigurations, or test transactions applied directly to a replica node. MySQL InnoDB Cluster Set prevents promotion of a replica cluster with errant transactions to ensure data consistency and avoid potential conflicts.

Resolution Steps

To resolve this issue, we inject the missing GTID into the Prod-site cluster without actually performing any data modification. This brings the GTID sets into alignment, allowing the promotion to proceed.

○ Step 01:

Set Variable for cluster Set

```
# var clusterSet=dba.getClusterSet()
```

○ Step 02:

Check the Cluster Set status with below command:

```
# clusterSet.status({extended: true})
```

This will show which '**transactionSetErrantGtidSet**' is absent on the primary site.

```
{
  "transactionSet": "2c2c1451-3090-11f0-bfed-0242ac160005:1-5,2ee4e196-309d-11f0-aa5a-0242ac160005:1-3:1000003-1000004,2ee4f461-309d-11f0-aa5a-0242ac160005:1-8,9209f4b5-308e-11f0-973c-0242ac160006:1-4,c2ea320-308f-11f0-bf60-0242ac160006:1-14064,c2eb7dd-308f-11f0-bf60-0242ac160006:1-12",
  "transactionSetConsistencyStatus": "INCONSISTENT",
  "transactionSetConsistencyStatusText": "There are 1 transactions that were executed in this instance that did not originate from the PRIMARY.",
  "transactionSetErrantGtidSet": "2ee4e196-309d-11f0-aa5a-0242ac160005:1000004",
  "transactionSetMissingGtidSet": ""
}
```

○ Step 03:

- Connect to a writable primary instance in the Prod-site cluster (e.g., mysql1:3306).
- Run the following SQL statements to inject the missing GTID as an empty transaction:

```
SET SESSION gtid_next = '2ee4e196-309d-11f0-aa5a-0242ac160005:1000004';
BEGIN;
COMMIT;
SET SESSION gtid_next = 'AUTOMATIC';
```

- Wait for the GTID to propagate through replication to all members of the Prod-site cluster.
- Confirm GTID consistency by running:

```
# SELECT @@GLOBAL.gtid_executed;
```

- Check the Cluster Set status again:

```
# clusterset.status()
```

```
MySQL mysql1:33060+ ssl JS > clusterset.status()  
{  
  "clusters": {  
    "DR-site": {  
      "clusterRole": "REPLICA",  
      "clusterSetReplicationStatus": "OK",  
      "globalStatus": "OK"  
    },  
    "Prod-site": {  
      "clusterRole": "PRIMARY",  
      "globalStatus": "OK",  
      "primary": "mysql1:3306"  
    }  
  },  
  "domainName": "POC_CS",  
  "globalPrimaryInstance": "mysql1:3306",  
  "primaryCluster": "Prod-site",  
  "status": "HEALTHY",  
  "statusText": "All clusters available."  
}
```

Additional Notes

This method ensures that no data is lost and no replication state is corrupted. It is a safe and recommended approach to handling isolated errant transactions when promoting DR clusters in MySQL InnoDB Cluster Set.